



UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT



Data Structure and Algorithm

Laboratory Activity No. 13

Tree Algorithm

Submitted by:
Regondola Jezreel P.

Instructor:
Engr. Maria Rizette H. Sayo

November 07, 2025

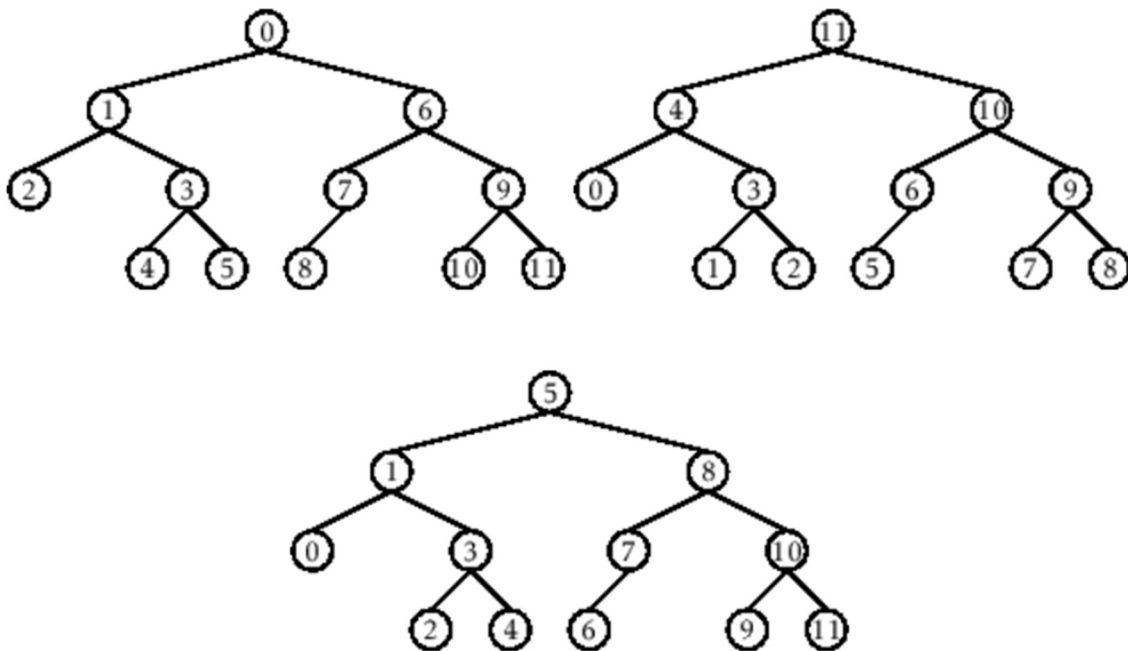
I. Objectives

Introduction

An abstract non-linear data type with a hierarchy-based structure is a tree. It is made up of links connecting nodes (where the data is kept). The root node of a tree data structure is where all other nodes and subtrees are connected to the root.

This laboratory activity aims to implement the principles and techniques in:

- To introduce Tree as Non-linear data structure
- To implement pre-order, in-order, and post-order of a binary tree



- Figure 1. Pre-order, In-order, and Post-order numberings of a binary tree

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Tree Implementation

```
class TreeNode:
```

```

def __init__(self, value):
    self.value = value
    self.children = []

def add_child(self, child_node):
    self.children.append(child_node)

def remove_child(self, child_node):
    self.children = [child for child in self.children if child != child_node]

def traverse(self):
    nodes = [self]
    while nodes:
        current_node = nodes.pop()
        print(current_node.value)
        nodes.extend(current_node.children)

def __str__(self, level=0):
    ret = " " * level + str(self.value) + "\n"
    for child in self.children:
        ret += child.__str__(level + 1)
    return ret

# Create a tree
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

print("Tree structure:")
print(root)

print("\nTraversal:")
root.traverse()

```

Questions:

- 1 When would you prefer DFS over BFS and vice versa?
- 2 What is the space complexity difference between DFS and BFS?

- 3 How does the traversal order differ between DFS and BFS?
- 4 When does DFS recursive fail compared to DFS iterative?

III. Results

```
Tree structure:
Root
  Child 1
    Grandchild 1
  Child 2
    Grandchild 2

Traversal:
Root
Child 2
Grandchild 2
Child 1
Grandchild 1
```

Figure 1 Screenshot of the program output

The program creates a tree with one root and two levels of children. The traverse method uses DFS (Depth-First Search) to visit nodes. It uses a stack that pops from the end, so it visits Child 2 before Child 1.

1. DFS is preferred when the goal is to go as deep as possible, such as in maze solving or pathfinding in a deep structure. BFS is better when we need the shortest path or the least number of steps, such as in social networks or routing problems.
2. DFS uses less memory because it only needs to store nodes in the current path, while BFS uses more memory since it keeps all nodes in the current level in the queue.
3. DFS visits nodes by going deep into one branch before moving to others, while BFS visits all nodes at the same level before going deeper.
4. DFS recursive may fail for very large graphs because it can exceed the recursion limit, while the iterative version avoids this problem by using a stack instead of recursion.

IV. Conclusion

This lab showed how to implement tree structures and traverse them using DFS. We learned that DFS explores depth-first and uses less memory than BFS. The recursive approach is simpler but fails on very deep trees, while iterative approach is safer for large data.

References

- [1] Co Arthur O.. "University of Caloocan City Computer Engineering Department Honor Code," UCC-CpE Departmental Policies, 2020.